Software Architecture & Design Specification (SAD)

Online Marketplace for Handcrafted Goods

Document Version: 1.0 (Baseline Architecture Release)
Project Allocation: Team 08 – Online Marketplace for Handcrafted Goods
Course Affiliation: Software Engineering Mini Project – Fall 2026
Date of Submission: 27-Oct-2026
Document Status: Submitted for Academic Review (Pending Evaluation / Approval)
Sprint 1 Duration: 27-Oct-2026 to 31-Oct-2026
Sprint 2 Duration: 02-Nov-2026 to 07-Nov-2026
Project Code Repository: https://github.com/krrishsingla11/SE-LAB-TEAM-08

Team Members & Roles

Roll Number	Student Name	Project Role	Core Architectural Ownership
PES1UG24AM141	Krrish Singla	Team Lead / Dev Lead	Layered Architecture, CI/CD Pipeline, Order & Payment Contracts
PES1UG24AM138	Kishan K Shenoy	QA Lead / Test Engineer	Architecture Verification, STRIDE Validation, Performance Testing
PES1UG24AM145	Kushal G	Backend & Database Engineer	Relational Schema Design, Concurrency & Locking, Data Persistence
PES1UG24AM171	Namith Kumar C	Frontend & UI/UX Engineer	Client SPA Architecture, REST API Consumption, UX Accessibility

Document Revision History

Version	Date	Author	Change Summary	Approval Status
0.1	14-Oct-2026	Team 08	Initial architectural sketch and technology stack evaluation	Draft Candidate

Version	Date	Author	Change Summary	Approval Status
0.9	25-Oct-2026	Kushal G & Krrish Singla	Defined component boundaries, sequence flows, and STRIDE model	Internal Review
1.0	27-Oct-2026	Team 08	Final SAD release with embedded visual UML diagrams and verified APIs	Submitted for Review (Pending Approval)

Document Review & Approval Sign-Off

Role	Name	Affiliation / Email	Sign-Off Status
Course Coordinator / Instructor	Chirag N. Bongale	PES University	Pending Review & Evaluation
Team Lead / Development Lead	Krrish Singla	PES1UG24AM141	Submitted by Author (27-Oct-2026)
Quality Assurance Lead	Kishan K Shenoy	PES1UG24AM138	Submitted by Author (27-Oct-2026)

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Intended Audience

1.4 Definitions and Acronyms

2. Document Overview

2.1 How to Use This Specification

2.2 Companion Project Deliverables

3. System Architecture

3.1 Architectural Goals & Quality Constraints

3.2 Stakeholders & Architectural Concerns

3.3 UML Component Diagram & Structural View (Landscape)

3.4 Component Descriptions & Module Responsibilities

3.5 Chosen Architecture Pattern & Rationale (ADR-001)

3.6 Technology Stack, Frameworks & Data Stores

3.7 Architectural Risks & Mitigation Strategies

3.8 Bidirectional Traceability to SRS Requirements

3.9 Security Architecture & STRIDE Threat Model

4. Detailed System Design

4.1 Design Overview & Modularity Principles

4.2 UML Sequence Diagrams (Visual Flows & Error Paths)

4.2.1 Sequence Flow 1: Buyer Checkout & Order Processing (Landscape)

4.2.2 Sequence Flow 2: Artisan Product Listing & Inventory Setup (Landscape)

4.3 Relational Data Model & Entity-Relationship Diagram (ERD)

4.3.1 Relational Schema Definitions (12 Normalized Entities)

4.3.2 Concurrency Control & Row-Level Locking Strategy

4.3.3 Visual ERD Diagram (Landscape)

4.4 RESTful API Design & Contract Specifications

4.5 Error Handling, Structured Logging & Health Monitoring

4.6 User Experience (UX) Architecture & Accessibility

4.7 CI/CD Pipeline & Automated Staging Plan

4.8 Open Issues & Future Enhancement Roadmap

5. Appendices

5.1 Architectural Glossary

5.2 References & Standards

5.3 Authoring Tools & Technologies

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) details the architectural decisions, structural decomposition, component interactions, relational data model, and API contracts for the "Online Marketplace for Handcrafted Goods". Developed by Team 08, it bridges the functional requirements set forth in the SRS with the implementation and verification work planned across Sprint 1 (27–31 Oct 2026) and Sprint 2 (2–7 Nov 2026).

1.2 Scope

The architectural scope encompasses the end-to-end software stack: client-side web SPA, RESTful API gateway and authentication middleware, domain business services (identity, catalog, cart, orders, payments, inventory, invoicing, and reporting), and the relational persistence engine. It also covers interfaces to external systems (payment gateway, SMTP email relay, GitHub Actions CI/CD).

1.3 Intended Audience

Software engineers (Team 08), QA test engineers, security auditors, and course evaluators.

1.4 Definitions and Acronyms

Acronym / Term	Definition & Architectural Relevance
ADR	Architecture Decision Record: formal capture of significant architectural design choices.
STRIDE	Threat analysis model: Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation.
ORM	Object-Relational Mapping: SQLAlchemy abstraction over relational database tables.
REST	Representational State Transfer: architectural style for distributed hypermedia systems.
JWT	JSON Web Token: cryptographically signed compact claims for authentication and RBAC.
ACID	Atomicity, Consistency, Isolation, Durability: relational transaction properties.
Row Lock	SELECT ... FOR UPDATE mechanism ensuring atomic inventory decrement under concurrency.
CI/CD	Continuous Integration / Continuous Deployment: GitHub Actions build and test automation.

2. Document Overview

2.1 How to Use This Specification

Section 3 (System Architecture) establishes the high-level macro architecture: component boundaries, layered patterns, ADR-001 architectural justification, STRIDE threat models, and requirement traceability. Section 4 (Detailed System Design) provides micro-architectural blueprints: sequence diagrams with alternate error paths, relational ERD schemas, row-level locking algorithms, exact API specifications, and logging strategies.

2.2 Companion Project Deliverables

- **SRS_Team08_OnlineMarketplace.docx / .pdf**: Defines baseline functional and non-functional requirements.
- **STP_Team08_OnlineMarketplace.docx / .pdf**: Establishes test scenarios, verification procedures, and acceptance criteria.
- **GitHub Repository (https://github.com/krrishsingla11/SE-LAB-TEAM-08)**: Houses version-controlled source code, docker-compose configurations, and CI/CD workflow definitions.

3. System Architecture

3.1 Architectural Goals & Quality Constraints

- **Strict Modularity**: High cohesion within domain services and low coupling between layers; services interact through typed Python interfaces.
- **Concurrency Safety & Zero Oversell**: Elimination of race conditions on inventory stock during simultaneous buyer checkouts via transactional row locks.
- **Security & Least Privilege**: Enforcement of TLS 1.2+, bcrypt password hashing, signed 24-hour JWT claims, and strict RBAC authorization.
- **Two-Sprint Feasibility**: Pragmatic architecture achievable by four undergraduate engineers within the announced course schedule without distributed system overhead.
- **Deterministic Containerization**: Complete reproducibility across local development machines and GitHub Actions CI/CD via Docker Compose.

3.2 Stakeholders & Architectural Concerns

Stakeholder Group	Primary Architectural Concerns & Quality Drivers
Buyers	Sub-second catalog browsing, reliable cart persistence, frictionless checkout, and instant PDF invoice retrieval.
Artisans (Sellers)	Simple product creation, automated inventory tracking, low-stock threshold alerts, and real-time sales reporting.
Platform Administrators	Centralized user role administration, dispute resolution visibility, and listing moderation capabilities.
Development Team (Team 08)	Clear component boundaries, unit-testable services with mockable DB sessions, and automated CI/CD builds.
Course Coordinator / Evaluator	Compliance with course guidelines, full SRS-to-SAD traceability, reproducible demonstration, and on-time sprint milestones.

3.3 UML Component Diagram & Structural Architecture View

Figure 1 provides a comprehensive UML Component Diagram representing the system architecture decomposed into distinct architectural layers: Client Presentation Layer, API Gateway & Security Layer, Business Logic Services Layer, Persistence Layer, and External Integrations. All communications enforce explicit interfaces.

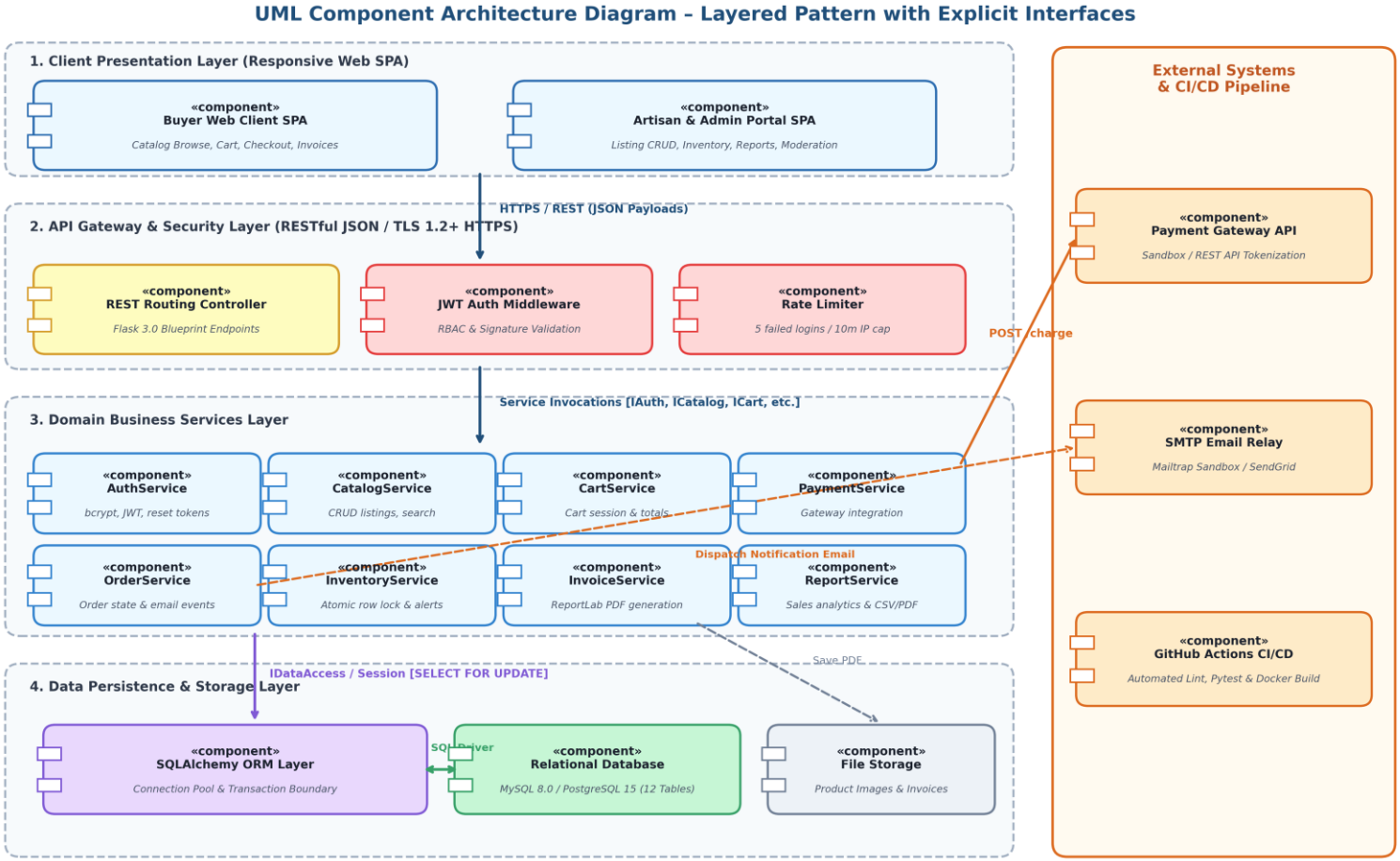


Figure 1: UML Component Architecture Diagram – Layered Pattern with Explicit Interfaces & Connectors

3.4 Component Descriptions & Module Responsibilities

Component Name	Core Functional Responsibilities	Direct Interface Collaborations
Client Web SPA	HTML5/CSS3/JavaScript responsive frontend delivering buyer storefronts, artisan portals, and admin consoles.	Web Router (REST/JSON over HTTPS)
Web Router / Controller	Flask/FastAPI routing engine parsing HTTP requests, serializing JSON responses, and managing error mapping.	JWT Middleware, Rate Limiter, Domain Services
JWT Auth Middleware	Intercepts HTTP Authorization headers, validates cryptographic signatures, checks 24h expiration, and extracts role claims.	AuthService, Web Router
Rate Limiter	Tracks authentication failure attempts per IP address in an in-memory bucket, triggering HTTP 429 when threshold exceeded.	Web Router, AuthService
AuthService	Manages user registration, credential hashing with bcrypt (cost 12), token generation, and password reset flows.	SQLAlchemy ORM (Users table), SMTP Relay
CatalogService	Governs handcrafted product creation, multi-image processing, metadata updates, soft deletions, and catalog search/filtering.	SQLAlchemy ORM (Products, Categories), File Storage
CartService	Maintains persistent shopping carts per authenticated buyer, calculates subtotals, and performs pre-checkout stock checks.	SQLAlchemy ORM (Carts, CartItems), InventoryService
PaymentService	Orchestrates payment charges via external gateway API (or mock stub in Sprint 1); ensures zero cardholder storage.	Payment Gateway API, OrderService
OrderService	Coordinates transactional order creation, tracks state progression (Pending->Processing->Shipped->Delivered), and dispatches buyer emails.	SQLAlchemy ORM (Orders, OrderItems), InvoiceService, SMTP
InventoryService	Enforces atomic stock decrements via row-level locks (SELECT ... FOR UPDATE) and issues low-stock alerts.	SQLAlchemy ORM (Inventory table)
InvoiceService	Compiles immutable PDF invoices using ReportLab containing line items, taxes, and totals; saves to local file storage.	File Storage, OrderService
ReportService	Aggregates sales revenue, units sold, and inventory metrics over configurable time periods; exports CSV and PDF.	SQLAlchemy ORM, InvoiceService
SQLAlchemy ORM	Provides object-relational mapping, connection pooling, and atomic transaction boundary management.	Relational Database (MySQL / PostgreSQL)
Relational Database	Stores persistent tabular data across 12 normalized tables with foreign key constraints and ACID guarantees.	SQLAlchemy ORM

3.4.1 Explicit Component Interface Contracts

To substantiate the explicit interface boundaries in Figure 1, inter-component interactions are governed by formal typed contracts. Components publish provided interfaces and consume required interfaces as detailed below:

Interface Contract	Provider	Consumers	Contract Method Signatures & Payload Expectations
IAuthContract	AuthService	Web Router, JWT Middleware	register_user(data), authenticate(email, pass), generate_jwt(user), verify_token(token), request_reset(email)
ICatalogContract	CatalogService	Web Router	create_product(artisan_id, data, imgs), update_product(id, data), search_catalog(query, cat, page)

Interface Contract	Provider	Consumers	Contract Method Signatures & Payload Expectations
ICartContract	CartService	Web Router, OrderService	get_cart(buyer_id), add_item(buyer_id, prod_id, qty), update_qty(item_id, qty), clear_cart(buyer_id)
IInventoryContract	InventoryService	OrderService, CatalogService	lock_and_validate_stock(items, db_sess), decrement_stock(items, db_sess), check_low_stock(prod_id)
IPaymentContract	PaymentService	OrderService	capture_charge(amount, token, idem_key), void_or_refund_charge(txn_ref, reason)
IOrderContract	OrderService	Web Router	checkout_cart(buyer_id, cart_id, addr, token), transition_status(order_id, new_status, artisan_id)
IInvoiceContract	InvoiceService	OrderService, Web Router	compile_pdf_invoice(order_id), stream_invoice_pdf(order_id, user_id)
IReportContract	ReportService	Web Router	aggregate_artisan_sales(artisan_id, period), export_report(artisan_id, format)
IRepositoryContract	SQLAlchemy ORM	All Domain Services	begin_transaction(), commit(), rollback(), execute_query(stmt), get_by_id(model, id)

3.5 Chosen Architecture Pattern & Rationale (ADR-001)

Architecture Decision Record: ADR-001 – Selection of Layered Monolithic Architecture

Status: APPROVED & ADOPTED

Date: 27-Oct-2026

Context: The project must be developed, tested, and demonstrated by a team of four undergraduate students within two tightly scheduled one-week sprints (Sprint 1: 27–31 Oct 2026; Sprint 2: 2–7 Nov 2026). The system domain encompasses e-commerce catalog discovery, shopping carts, transactional checkout, inventory locking, PDF invoicing, and analytics.

Evaluated Architectural Alternatives:

- **Alternative 1: Microservices Architecture**: Decomposing the platform into 6+ autonomous container services (AuthService, CatalogService, CartService, OrderService, InventoryService, NotificationService) communicating via gRPC or message queues. REJECTED due to disproportionate operational overhead: requires service mesh, distributed transaction coordination (Saga pattern), complex network observability, and multi-repo CI/CD orchestration that would severely endanger the 2-sprint project schedule.
- **Alternative 2: Serverless Functions (FaaS)**: Implementing each endpoint as an AWS Lambda / Cloud Function. REJECTED due to cold-start execution latencies, localized database connection pooling exhaustion during concurrency spikes, local debugging friction, and difficulties packaging complex PDF generation binaries.
- **Alternative 3: Layered (N-Tier) Monolithic Architecture**: Structuring the application as a unified codebase with rigorously isolated architectural layers (Presentation, Routing/Security, Business Domain Services, and Relational Persistence). SELECTED AND APPROVED.

Rationale: The Layered Monolith delivers the optimal balance of development velocity, simplicity, and architectural cleanliness for Team 08. In-process function invocations eliminate inter-service network latency and serialization failures. ACID transactions across orders and inventory can be executed atomically via database-level transaction scopes without distributed two-phase commit overhead. Furthermore, clean modular separation ensures individual service classes can be unit-tested with mocked ORM sessions, and provides a straightforward migration path toward microservices if future scale demands.

3.6 Technology Stack, Frameworks & Data Stores

Architectural Layer	Technology & Version Selection	Engineering Rationale
Client Frontend	HTML5, CSS3, Modern ES6+ JavaScript, Tailwind CSS	Lightweight, rapid UI prototyping, zero build-tool bloat for sprint timeline.
Web / API Framework	Python 3.12 with Flask 3.0 / FastAPI	High productivity, mature ecosystem, native JSON serialization, simple middleware.
ORM / Data Layer	SQLAlchemy 2.0 with Alembic Migrations	Enterprise-grade ORM, database-agnostic abstractions, robust transaction controls.
Relational Database	MySQL 8.0 / PostgreSQL 15 (12 Normalized Tables)	Full ACID compliance, support for row-level locking (SELECT ... FOR UPDATE).
PDF Generation	ReportLab 4.1 (Pure-Python Engine)	Deterministic server-side PDF/A compilation, no external headless browser needed.

Architectural Layer	Technology & Version Selection	Engineering Rationale
Authentication	Flask-JWT-Extended / PyJWT with bcrypt	Cryptographically secure password hashing (cost 12) and stateless JWT tokens.
Containerization	Docker 26.x & Docker Compose 2.x	Reproducible multi-container stack across local development and GitHub Actions.
CI/CD Engine	GitHub Actions (Ubuntu-latest runner)	Automated linting, pytest test execution, and docker container health checks on PR.
Testing Tools	pytest, Postman, Apache JMeter 5.6, OWASP ZAP	Comprehensive coverage of unit, integration, load, and security testing.

3.7 Architectural Risks & Mitigation Strategies

Identified Architectural Risk	Severity	Potential Failure Scenario	Impact	Architectural Mitigation Strategy
Inventory Race Conditions	High	Multiple buyers checking out the final handcrafted item simultaneously leading to overselling.	High	Enforce database row-level locking (SELECT ... FOR UPDATE) inside InventoryService within a strict database transaction scope.
Payment Gateway Availability	Medium	External sandbox API downtime stalling development or testing during sprint evaluation.	High	Implement a robust mock payment stub in Sprint 1 with configurable success/decline behaviors, switching to gateway sandbox in Sprint 2.
Schema Drift Across Sprints	Medium	Database schema inconsistencies between developer machines causing unrepeatable test failures.	High	Enforce version-controlled database migrations using Alembic integrated into Docker entrypoint scripts.
Slow PDF Generation Latency	Low	ReportLab document compilation blocking WSGI request threads during bulk invoice downloads.	Medium	Cache generated PDF invoices to local file storage upon order completion; subsequent requests stream the cached static file directly.
Sprint Scope Overrun	High	Attempting complex features causing incomplete deliverables by the 7 Nov 2026 dev freeze.	High	Strict Jira backlog prioritization; decouple non-essential features into Sprint 2 stretch goals; maintain daily standup alignment.

3.8 Bidirectional Traceability to SRS Requirements

SRS Requirement Group	Functional Scope	Architectural Implementation Components
OMH-F-001 to OMH-F-004	Identity & Access Management	AuthService, JWT Auth Middleware, Rate Limiter, SMTP Relay
OMH-F-005 to OMH-F-007	Product Catalog & Search	CatalogService, File Storage, ORM (Products, Categories, ProductImages)
OMH-F-008 to OMH-F-009	Shopping Cart & Pre-Checkout	CartService, InventoryService, ORM (Carts, CartItems tables)
OMH-F-010 to OMH-F-011	Payment Gateway Integration	PaymentService, Payment Gateway API, OrderService
OMH-F-012 to OMH-F-014	Order Processing & Tracking	OrderService, SMTP Relay, ORM (Orders, OrderItems tables)
OMH-F-015 to OMH-F-016	Inventory Locking & Stock Alerts	InventoryService (Row-Level Lock), ORM (Inventory table)
OMH-F-017 to OMH-F-018	Server-Side PDF Invoicing	InvoiceService (ReportLab Engine), File Storage, OrderService
OMH-F-019 to OMH-F-020	Artisan Sales Reporting	ReportService, ORM Aggregation Queries, CSV/PDF Exporter
OMH-NF-001 to OMH-NF-005	Performance, Scalability, A11y	Web Layer, SQLAlchemy Connection Pool, Tailwind A11y Themes
OMH-SR-001 to OMH-SR-005	Security, Passwords, Logging	bcrypt Engine, JWT Validator, Rate Limiter, Structured JSON Logger

3.9 Security Architecture & STRIDE Threat Model

STRIDE Threat Category	Marketplace Attack Scenario	Architectural Defense & Mitigation Mechanism
Spoofing	Attacker impersonates an Artisan to alter another seller's item pricing or hijack sales revenue.	Every protected API route verifies the cryptographic HMAC signature of the JWT and checks that artisan_id matches product.seller_id.
Tampering	Malicious buyer modifies client-side HTTP payload to checkout with price = $0.01 instead of catalog price.	OrderService recalculates item prices and subtotal amounts exclusively from server-side database records, ignoring client totals.
Repudiation	Artisan claims they never received or fulfilled an order; buyer claims order was never placed.	Immutable database records with UTC timestamps and audit logs track all order transitions, payment refs, and fulfillment status updates.
Information Disclosure	Eavesdropper intercepts network traffic to steal passwords, session tokens, or customer addresses.	Mandatory TLS 1.2+ HTTPS encryption on all traffic; passwords hashed with bcrypt (cost 12); strict log sanitization excludes tokens.
Denial of Service	Attacker executes automated credential stuffing or floods search endpoints with expensive wildcards.	RateLimiter component restricts failed logins to <= 5 per 10 min per IP; database queries enforce strict pagination limits (max 50/page).
Elevation of Privilege	Standard Buyer crafts HTTP requests attempting to access /api/admin/users or execute moderation actions.	JWT Auth Middleware validates the role claim in token payload; administrative endpoints explicitly require role == "admin".

4. Detailed System Design

4.1 Design Overview & Modularity Principles

The detailed design adheres strictly to Domain-Driven Design (DDD) principles adapted for a modular Python monolith. Each domain service exposes deterministic method contracts, encapsulates business invariant checks, and communicates with data tables exclusively via SQLAlchemy models.

4.2.1 Sequence Flow 1: Atomic Buyer Checkout Flow with Pre-Payment Stock Lock

Figure 2 presents the complete sequence interaction for the buyer checkout journey. CRITICAL DESIGN PRINCIPLE: Stock availability is atomically locked and reserved BEFORE any payment is captured. If stock is depleted (Alt 1), checkout aborts immediately with HTTP 400 without charging the buyer. If payment authorization fails (Alt 2), the database transaction rolls back, releasing all reserved stock locks, and returns HTTP 402.

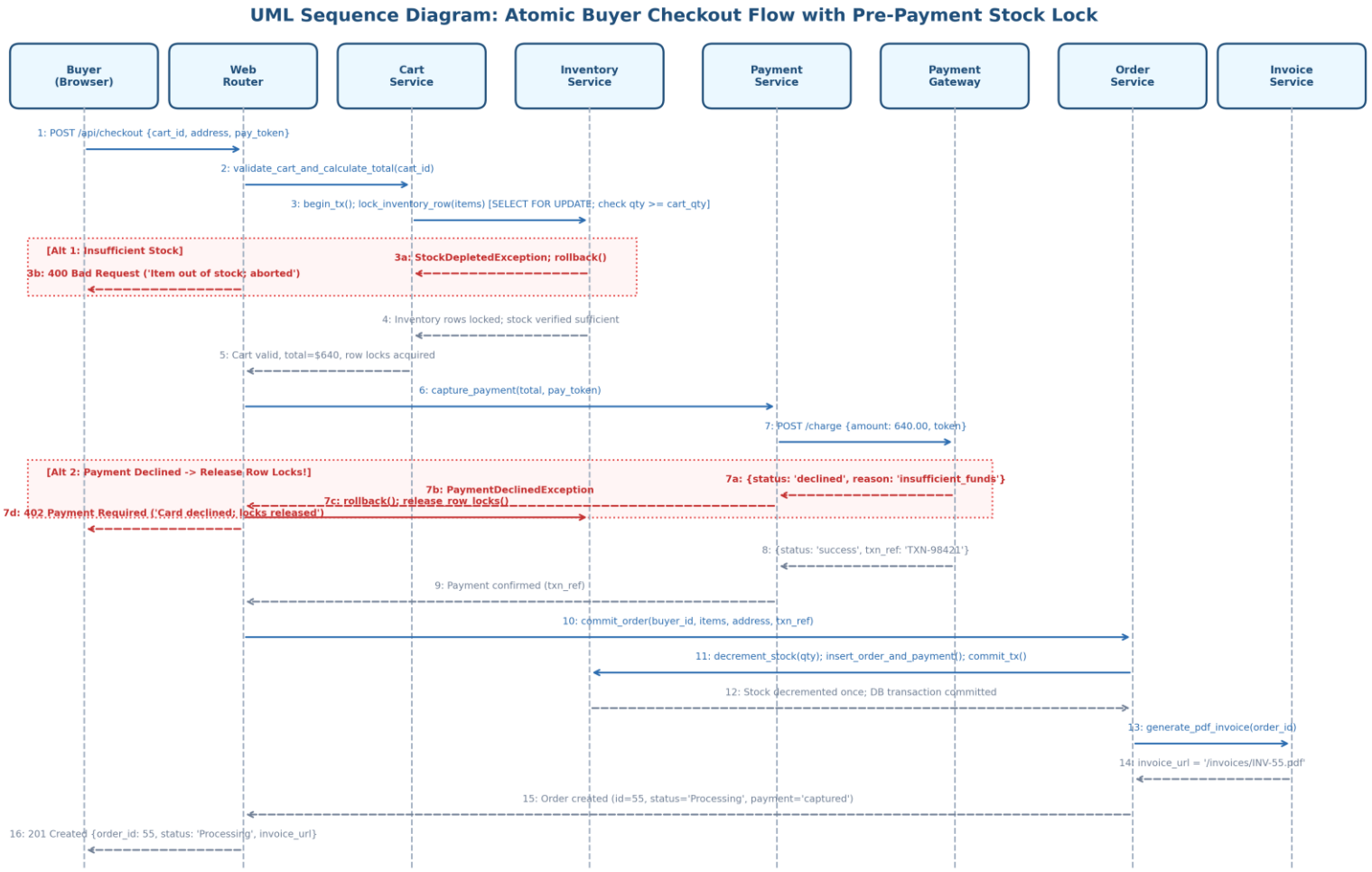


Figure 2: UML Sequence Diagram – Buyer Checkout Flow with Atomic Pre-Payment Stock Reservation & Rollback Paths

4.2.2 Sequence Flow 2: Artisan Product Listing Creation & Inventory Setup

Figure 3 depicts the sequence flow for an artisan creating and publishing a new handcrafted product. It explicitly models token role verification (Alt 1: Unauthorized access returning HTTP 403), payload validation (Alt 2: Validation failure returning HTTP 422), multi-image file storage persistence, and atomic inventory record initialization.

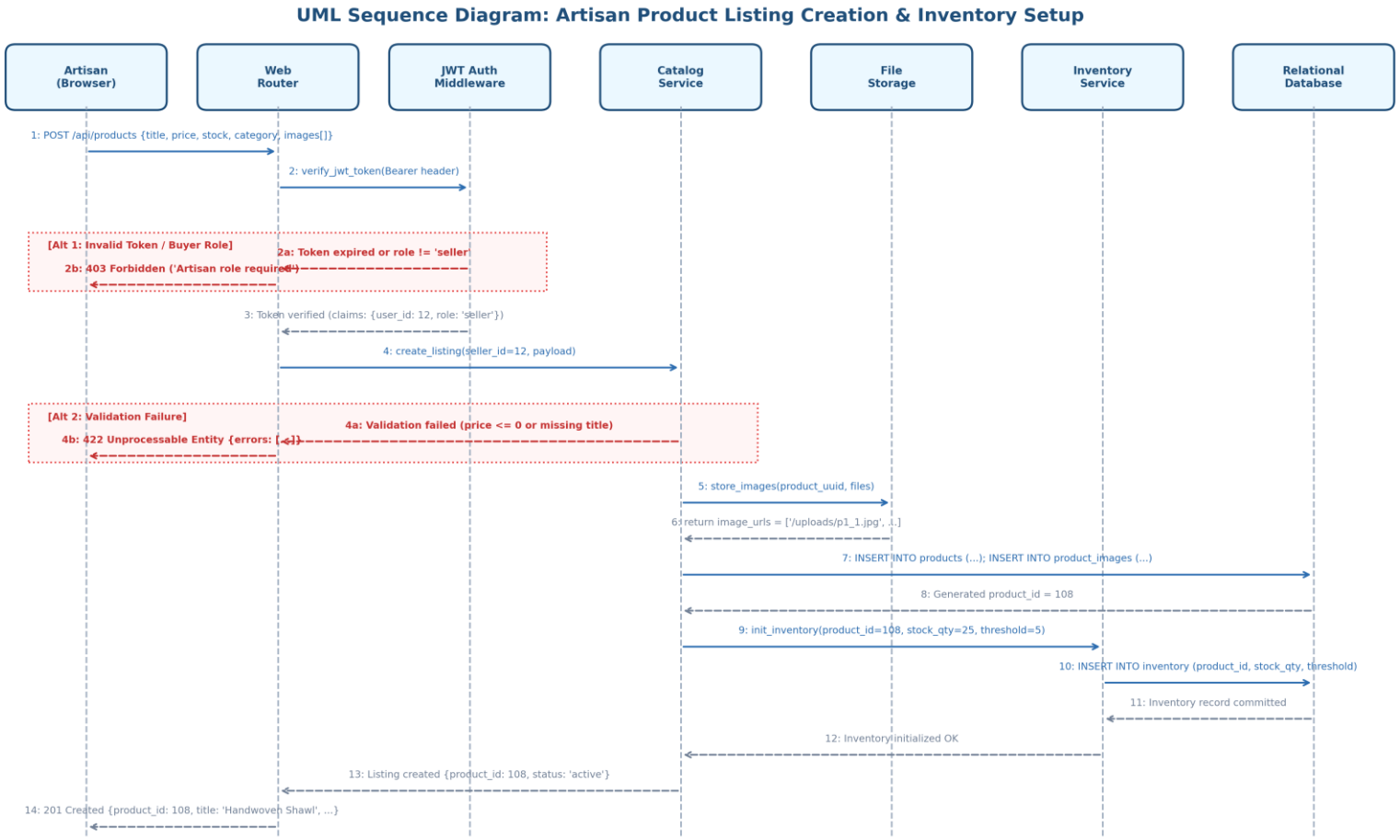


Figure 3: UML Sequence Diagram – Artisan Product Creation, Multi-Image Storage & Inventory Initialization

4.3 Relational Data Model & Entity-Relationship Diagram (ERD)

Figure 4 illustrates the complete relational database schema supporting all marketplace operations. The schema encompasses exactly 12 normalized entities connected by strict foreign key constraints. Notice that CART_ITEMS relates to CARTS and PRODUCTS (no direct user link); ORDER_ITEMS relates to ORDERS and PRODUCTS. PRODUCT_IMAGES supports multiple showcase photos per handcrafted craft item.

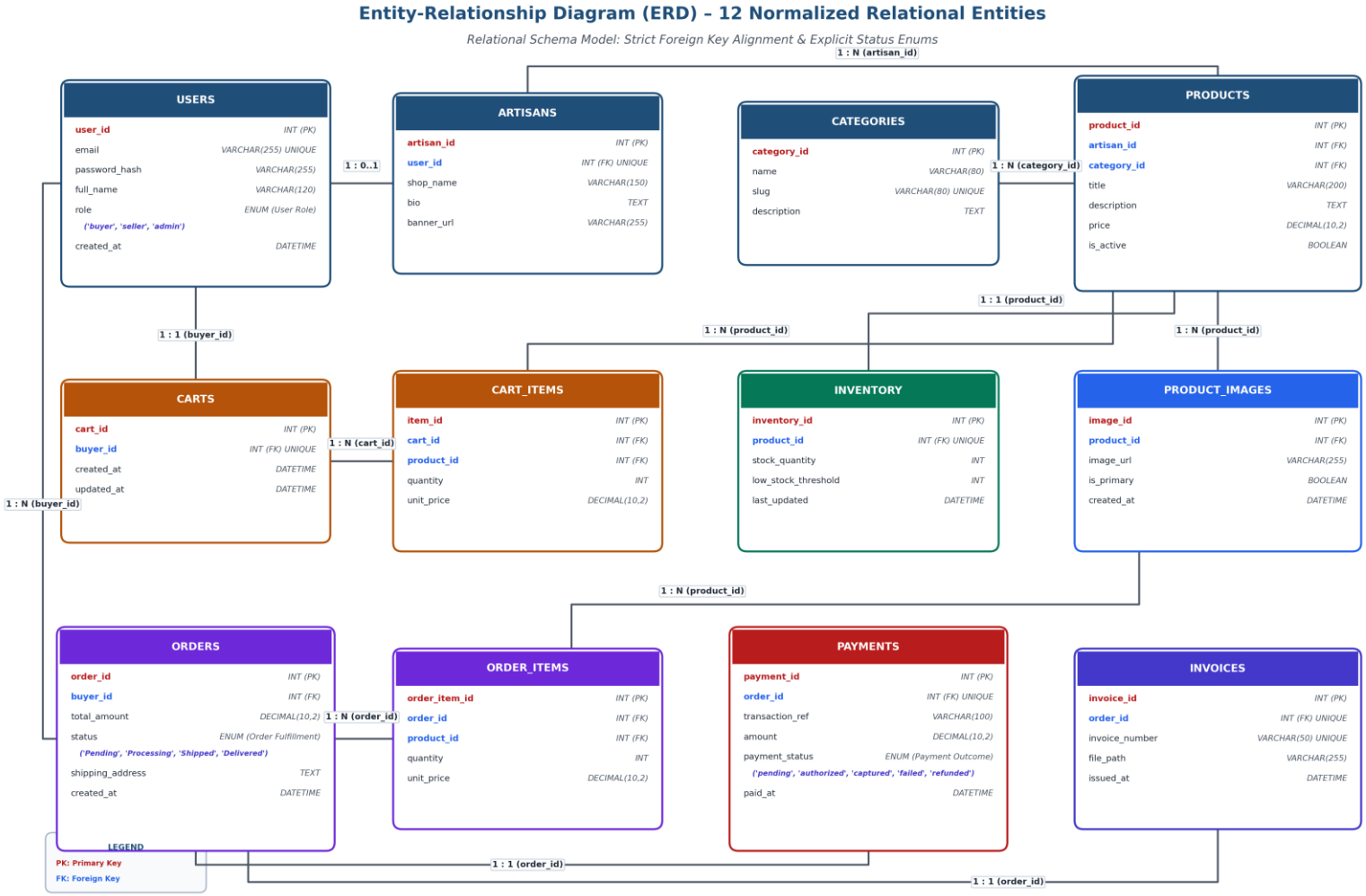


Figure 4: Entity-Relationship Diagram (ERD) – 12 Normalized Relational Entities & Clean Foreign Key Constraints

4.3.1 Relational Schema Definitions (12 Normalized Entities)

The marketplace persistence layer consists of exactly 12 normalized tables:

1. USERS: Primary account identity (user_id PK, email, password_hash, full_name, role, created_at).
2. ARTISANS: Artisan profile details (artisan_id PK, user_id FK UNIQUE, shop_name, bio, banner_url).
3. CATEGORIES: Handcrafted product taxonomies (category_id PK, name, slug UNIQUE, description).
4. PRODUCTS: Craft item listings (product_id PK, artisan_id FK, category_id FK, title, description, price, is_active).
5. PRODUCT_IMAGES: Multiple high-resolution images per craft (image_id PK, product_id FK, image_url, is_primary, created_at).
6. INVENTORY: Concurrency-locked stock ledger (inventory_id PK, product_id FK UNIQUE, stock_quantity, low_stock_threshold, last_updated).
7. CARTS: Persistent customer shopping cart (cart_id PK, buyer_id FK UNIQUE, created_at, updated_at).
8. CART_ITEMS: Line items in cart (item_id PK, cart_id FK, product_id FK, quantity, unit_price).
9. ORDERS: Finalized orders (order_id PK, buyer_id FK, total_amount, status ENUM('Pending','Processing','Shipped','Delivered'), shipping_address, created_at).
10. ORDER_ITEMS: Line items in order (order_item_id PK, order_id FK, product_id FK, quantity, unit_price).
11. PAYMENTS: Financial transaction ledger (payment_id PK, order_id FK UNIQUE, transaction_ref, amount, payment_status ENUM('pending','authorized','captured','failed','refunded'), paid_at).
12. INVOICES: Server-generated PDF records (invoice_id PK, order_id FK UNIQUE, invoice_number UNIQUE, file_path, issued_at).

4.3.2 Concurrency Control, Atomic Locking & Checkout Transaction Model

To eliminate inventory race conditions during concurrent checkouts of the same last unit and guarantee zero overselling, the marketplace executes a single coherent database transaction with pessimistic row-level locking via SQLAlchemy:

```
SELECT stock_quantity FROM inventory WHERE product_id = :id FOR UPDATE;
```

The transaction sequence is strictly orchestrated as follows:

1. Transaction Lease & Lock: When a buyer requests checkout (POST /api/checkout), the backend begins a database transaction and executes SELECT stock_quantity FROM inventory WHERE product_id = :id FOR UPDATE on each line item. This acquires an exclusive database row lock, queuing concurrent checkout attempts for the same item.
2. Availability Validation: The service verifies stock_quantity >= cart_item.quantity. Crucially, inventory is NOT decremented at this stage; the row lock simply protects and validates the available quantity.
3. Insufficient Stock Handling (Alt 1): If any item has insufficient stock, the transaction is immediately rolled back (ROLLBACK), releasing all locks with zero inventory change, no payment attempt, and no order creation. The client receives HTTP 400 Bad Request ("Item out of stock").
4. Payment Capture: With the database row lock held, PaymentService calls the external Payment Gateway (or mock stub). If the payment is declined or times out (Alt 2), the database transaction is immediately rolled back (ROLLBACK). All row locks are released, leaving inventory intact with zero residual reduction, and the client receives HTTP 402 Payment Required ("Card declined; locks released").
5. Single Stock Decrement & Atomic Commitment: Upon payment confirmation (HTTP 200 with gateway

txn_ref), the service executes exactly ONE stock decrement per order line (UPDATE inventory SET stock_quantity = stock_quantity - :qty), inserts the record into ORDERS (status="Processing"), inserts line items into ORDER_ITEMS, inserts the audit ledger record into PAYMENTS (payment_status="captured", transaction_ref=txn_ref), and executes COMMIT. Only after a successful database commit does InvoiceService compile the PDF invoice and dispatch confirmation emails.

6. Failure Recovery & Reconciliation: If payment capture succeeds at the gateway but the subsequent database commit fails (e.g. database connection partition), the service catches the database exception, logs an alert with the orphan txn_ref, and invokes an automated compensating call POST /api/gateway/void or /refund to void or refund the charge. An auditable reconciliation log entry is written for operational review.

7. Architectural Trade-offs & Limitations: Holding the row lock during the external payment roundtrip guarantees zero overselling without distributed locks, but limits concurrency throughput during high load. To mitigate blocking, the payment gateway client enforces a strict 3.0-second HTTP timeout, and the database connection pool is sized to 20 connections.

4.4 RESTful API Design & Contract Specifications

All endpoints exchange JSON over HTTPS and enforce HTTP status codes. Protected routes require HTTP header:

Authorization: Bearer <jwt-token>

Method	API Endpoint	Request Body / Parameters	Response & Error Payloads	Auth Scope	SRS Ref
POST	/api/auth/register	{"email", "password", "full_name", "role"}	201 Created {"user_id", "email"} 409 Conflict (Duplicate email)	Public	OMH-F-001
POST	/api/auth/login	{"email", "password"}	200 OK {"token", "expires_in": 86400} 401 Unauthorized (Invalid credentials)	Public	OMH-F-002
POST	/api/auth/reset-password	{"email"}	200 OK {"message": "Reset email dispatched"} 404 Not Found (Email not registered)	Public	OMH-F-004
GET	/api/products	Query: ?q=silk&category=2&page=1	200 OK {"products": [...], "total": 42, "page": 1}	Public	OMH-F-007
POST	/api/products	{"title", "desc", "price", "stock_qty", "category_id"}	201 Created {"product_id": 108} 403 Forbidden (Not artisan) 422 Unprocessable	Bearer (Artisan)	OMH-F-005
PUT	/api/products/{id}	{"price": 320.00, "stock_qty": 15}	200 OK {"product_id": 108, "price": 320.00} 403 Forbidden (Non-owner)	Bearer (Artisan)	OMH-F-006
POST	/api/cart/items	{"product_id": 108, "quantity": 2}	200 OK {"cart": [...], "subtotal": 640.00} 400 Bad Request (Stock exceeded)	Bearer (Buyer)	OMH-F-008
POST	/api/checkout	{"shipping_address", "payment_token"}	201 Created {"order_id": 55, "status": "Processing", "payment_status": "captured", "invoice_url": "..."} 400 Out of Stock 402 Declined (Locks Released)	Bearer (Buyer)	OMH-F-010, F-012
GET	/api/orders/{id}/invoice	Path: order_id	200 OK (Content-Type: application/pdf) 403 Forbidden (Non-owner) 404 Not Found	Bearer (Buyer/Artisan)	OMH-F-017, F-018
PUT	/api/orders/{id}/status	{"status": "Shipped"}	200 OK {"order_id": 55, "status": "Shipped"} 400 Bad Transition 403 Non-owner	Bearer (Artisan)	OMH-F-013
GET	/api/reports/sales	Query: ?period=monthly	200 OK {"revenue": 4500.00, "units": 28} 403 Forbidden (Not artisan)	Bearer (Artisan)	OMH-F-019
GET	/api/reports/export	Query: ?format=csv|pdf	200 OK (Content-Type: text/csv or application/pdf)	Bearer (Artisan)	OMH-F-020

4.5 Error Handling, Structured Logging & Health Monitoring

Standard Error Response Schema:

All error conditions return a consistent JSON structure:


```
{
  "error": "INSUFFICIENT_STOCK",
  "message": "The requested item 'Ceramic Vase' only has 1 unit available.",
  "status_code": 400,
  "timestamp": "2026-10-27T14:30:00Z"
}
```


Structured JSON Logging Strategy:

Application events are written to stdout in structured JSON format:

{"timestamp":"2026-10-27T14:30:01Z","level":"INFO","service":"OrderService","action":"ORDER_CREATED","order_id":55,"buyer_id":12,"amount":640.00}

Sensitive parameters (passwords, JWT secrets, raw card tokens) are automatically filtered via log scrubbing rules.

Health Monitoring Endpoint:

GET /api/health returns HTTP 200 with database connectivity status and container uptime metrics for Docker health-checks.

4.6 User Experience (UX) Architecture & Accessibility

- **Mobile-First Responsive Layout**: Breakpoints configured at 640px, 768px, 1024px, and 1280px ensuring effortless navigation across phones, tablets, and wide monitors.
- **WCAG 2.1 AA Compliance**: Color contrast ratio exceeds 4.5:1 for all text; semantic HTML5 elements (nav, main, section, footer) enable screen readers.
- **Visual Feedback & Affordance**: Skeleton loading states during catalog fetching; immediate badge updates on shopping cart icon; modal confirmations for deletions.

4.7 CI/CD Pipeline & Automated Staging Plan

The project utilizes GitHub Actions (.github/workflows/ci.yml) triggered on every push and pull request. The pipeline executes in three stages: (1) Static Analysis with flake8 linting and formatting; (2) Automated Test Suite with pytest running unit and integration tests against a test database; and (3) Docker Build & Smoke Test building the image and verifying container health against /api/health.

4.8 Open Issues & Future Enhancement Roadmap

- **Sprint 2 Payment Transition**: Replace the deterministic local payment stub with live sandbox credentials.
- **Cloud Object Storage Migration**: Transition product images from local container volumes to Amazon S3 / MinIO for production deployments.
- **Artisan Direct Messaging**: Future enhancement to enable direct buyer-artisan communication regarding custom craft orders.

5. Appendices

5.1 Architectural Glossary

- **ACID**: Atomicity, Consistency, Isolation, Durability guarantees enforced by relational database transactions.
- **Pessimistic Lock**: Concurrency control mechanism that locks a database record at read time until transaction commit.
- **Tokenization**: Replacing sensitive card data with an opaque, mathematically non-reversible string generated by payment gateways.

5.2 References & Standards

- **IEEE 42010-2011**: Systems and Software Engineering – Architecture Description.
- **OWASP Top 10 (2021)**: Standard awareness document for web application vulnerabilities.
- **NIST SP 800-160**: Systems Security Engineering: Considerations for a Multidisciplinary Approach.

5.3 Authoring Tools & Technologies

- **Matplotlib / Python Imaging**: Rendered publication-grade UML Component, Sequence, and ERD diagrams.
- **Python-docx & ReportLab**: Automated document authoring, formatting, and PDF compilation.